

# CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>

#define MAX_HISTORY 20
#define INITIAL_BUFFER_SIZE 64

char *history[MAX_HISTORY];
int history_count = 0;

struct termios original_terminal;

/* Restore normal terminal settings */
void disable_raw_mode()
{
    tcsetattr(STDIN_FILENO, TCSAFLUSH, &original_terminal);
}

/* Enable character-by-character input */
void enable_raw_mode()
{
    tcgetattr(STDIN_FILENO, &original_terminal);

    atexit(disable_raw_mode);

    struct termios raw = original_terminal;

    raw.c_lflag &= ~(ICANON | ECHO);

    raw.c_cc[VMIN] = 1;
    raw.c_cc[VTIME] = 0;

    tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}

/* Store command in history */
void add_history(const char *command)
{
    if (strlen(command) == 0)
        return;

    if (history_count == MAX_HISTORY)
    {
        free(history[0]);

        for (int i = 1; i < MAX_HISTORY; i++)
            history[i - 1] = history[i];
    }
}
```

```

        history_count--;
    }

    history[history_count] = strdup(command);
    history_count++;
}

/* Display history */
void show_history()
{
    printf("\n\nCommand History:\n");

    for (int i = 0; i < history_count; i++)
    {
        printf("%d %s\n", i + 1, history[i]);
    }
}

/* Clear the current input line */
void clear_line(int length)
{
    for (int i = 0; i < length; i++)
    {
        printf("\b \b");
    }
}

/* Read command with arrow-key navigation */
char *read_command()
{
    size_t buffer_size = INITIAL_BUFFER_SIZE;
    char *buffer = malloc(buffer_size);

    if (buffer == NULL)
    {
        perror("malloc");
        exit(EXIT_FAILURE);
    }

    size_t length = 0;
    int history_position = history_count;

    buffer[0] = '\0';

    printf("Command> ");
    fflush(stdout);

    while (1)
    {
        char c;

        if (read(STDIN_FILENO, &c, 1) != 1)
            break;

```

```

/* Enter key */
if (c == '\n' || c == '\r')
{
    printf("\n");
    break;
}

/* Backspace */
else if (c == 127 || c == '\b')
{
    if (length > 0)
    {
        length--;
        buffer[length] = '\0';

        printf("\b\b");
        fflush(stdout);
    }
}

/* Escape sequence */
else if (c == 27)
{
    char sequence[2];

    if (read(STDIN_FILENO, &sequence[0], 1) != 1)
        continue;

    if (read(STDIN_FILENO, &sequence[1], 1) != 1)
        continue;

    /* UP arrow: ESC [ A */
    if (sequence[0] == '[' && sequence[1] == 'A')
    {
        if (history_position > 0)
        {
            history_position--;

            clear_line(length);

            strcpy(buffer, history[history_position]);
            length = strlen(buffer);

            printf("%s", buffer);
            fflush(stdout);
        }
    }

    /* DOWN arrow: ESC [ B */
    else if (sequence[0] == '[' && sequence[1] == 'B')
    {
        if (history_position < history_count - 1)
        {
            history_position++;

```

```

        clear_line(length);

        strcpy(buffer, history[history_position]);
        length = strlen(buffer);

        printf("%s", buffer);
        fflush(stdout);
    }
    else
    {
        history_position = history_count;

        clear_line(length);

        length = 0;
        buffer[0] = '\0';

        fflush(stdout);
    }
}

/* Normal character */
else
{
    /* Resize buffer when necessary */
    if (length + 1 >= buffer_size)
    {
        buffer_size *= 2;

        char *new_buffer = realloc(buffer, buffer_size);

        if (new_buffer == NULL)
        {
            free(buffer);
            perror("realloc");
            exit(EXIT_FAILURE);
        }

        buffer = new_buffer;
    }

    buffer[length++] = c;
    buffer[length] = '\0';

    putchar(c);
    fflush(stdout);
}
}

return buffer;
}

```

```

int main()
{
    enable_raw_mode();

    printf("Interactive Command History Program\n");
    printf("Use UP/DOWN arrows to navigate history.\n");
    printf("Type 'history' to display commands.\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        char *command = read_command();

        if (strcmp(command, "exit") == 0)
        {
            free(command);
            break;
        }

        if (strcmp(command, "history") == 0)
        {
            show_history();
            free(command);
            continue;
        }

        if (strlen(command) > 0)
        {
            add_history(command);
            printf("You entered: %s\n", command);
        }

        free(command);
    }

    /* Free stored history */
    for (int i = 0; i < history_count; i++)
    {
        free(history[i]);
    }

    printf("\nProgram terminated.\n");

    return 0;
}

```

# Screen shots:

```
sohan_jadala@DESKTOP-OI7PGP2:~/2520080060_OSSP$ cd 2520080060_Skill
sohan_jadala@DESKTOP-OI7PGP2:~/2520080060_OSSP/2520080060_Skill$ gedit Skill_3.c

** (gedit:4043): WARNING **: 09:45:42.895: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found

** (gedit:4043): WARNING **: 09:45:42.895: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
sohan_jadala@DESKTOP-OI7PGP2:~/2520080060_OSSP/2520080060_Skill$ gcc Skill_3.c -o Skill_3
sohan_jadala@DESKTOP-OI7PGP2:~/2520080060_OSSP/2520080060_Skill$ ./Skill_3
Command History Program
Type commands and press ENTER.
Use UP/DOWN arrows to recall commands.
Type 'history' to display history.
Type 'exit' to quit.

> hello
Command entered: hello
> operating system
Command entered: operating system
> kernal
Command entered: kernal
> history

Command History:
1 hello
2 operating system
3 kernal
> exit
sohan_jadala@DESKTOP-OI7PGP2:~/2520080060_OSSP/2520080060_Skill$ |
```